

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и Компьютерной техники

Лабораторная работа №4
Аппроксимация функции методом наименьших квадратов
Вариант 5

Выполнила:
Косогова Мария Александровна
Группа: Р3206
Проверил:
Зинченко Антон Андреевич,
Преподаватель практики.

Санкт-Петербург 2026

Содержание

ЗАДАНИЕ	3
ОСНОВНЫЕ ЭТАПЫ ВЫПОЛНЕНИЯ.....	8
1. ВЫЧИСЛИТЕЛЬНАЯ РЕАЛИЗАЦИЯ ЗАДАЧИ	8
2. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ЗАДАЧИ.....	11
ВЫВОД.....	24

Задание

$$y = \frac{6x}{x^4 + 5}, x \in [0; 2]; h = 0,2$$

1. Вычислительная реализация задачи

- 1) Сформировать таблицу табулирования заданной функции на указанном интервале (см. табл. 1)
- 2) Построить линейное и квадратичное приближения по 11 точкам заданного интервала;
- 3) Найти среднеквадратические отклонения для каждой аппроксимирующей функции. Ответы дать с тремя знаками после запятой;
- 4) Выбрать наилучшее приближение;
- 5) Построить графики заданной функции, а также полученные линейное и квадратичное приближения.

2. Программная реализация задачи

Для исследования использовать:

- линейную функцию,
- полиномиальную функцию 2-й степени,
- полиномиальную функцию 3-й степени,
- экспоненциальную функцию,
- логарифмическую функцию,
- степенную функцию.

Методика проведения исследования:

- 1) Вычислить меру отклонения: $S = \sum_{i=1}^n [\varphi(x_i) - y_i]^2$ для всех исследуемых функций;
- 2) Уточнить значения коэффициентов эмпирических функций, минимизируя функцию S;
- 3) Сформировать массивы предполагаемых эмпирических зависимостей $(\varphi(x_i), \varepsilon_i)$;
- 4) Определить среднеквадратичное отклонение для каждой аппроксимирующей функции. Выбрать наименьшее значение и, следовательно, наилучшее приближение;
- 5) Построить графики полученных эмпирических функций.

Задание:

- 1) Предусмотреть ввод исходных данных из файла/консоли (таблица $y = f(x)$ должна содержать от 8 до 12 точек);

- 2) Реализовать метод наименьших квадратов, исследуя все указанные функции;
- 3) Предусмотреть вывод результатов в файл/консоль: коэффициенты аппроксимирующих функций, среднеквадратичное отклонение, массивы значений $x_i, y_i, \varphi(x_i), \varepsilon_i$;
- 4) Для линейной зависимости вычислить коэффициент корреляции Пирсона;
- 5) Вычислить коэффициент детерминации, программа должна выводить соответствующее сообщение в зависимости от полученного значения R^2 ;
- 6) Программа должна отображать наилучшую аппроксимирующую функцию;
- 7) Организовать вывод графиков функций, графики должны полностью отображать весь исследуемый интервал (с запасом);
- 8) Программа должна быть протестирована при различных наборах данных, в том числе и некорректных;

Формулы

- Суммы:

$$SX = \sum_{i=1}^n x_i$$

$$SXX = \sum_{i=1}^n x_i^2$$

$$SX^3 = \sum_{i=1}^n x_i^3$$

$$SX^4 = \sum_{i=1}^n x_i^4$$

$$SY = \sum_{i=1}^n y_i$$

$$SXY = \sum_{i=1}^n x_i y_i$$

$$SX^2Y = \sum_{i=1}^n x_i^2 y_i$$

- Линейная аппроксимация:

$$P_1(x) = ax + b$$

- Система:

$$\begin{cases} a \cdot SX + b \cdot SX = SXY \\ a \cdot SX + b \cdot n = SY \end{cases}$$

- Метод Крамера:

$$\Delta = SXX \cdot n - SX \cdot SX$$

$$\Delta_1 = SXY \cdot n - SX \cdot SY$$

$$\Delta_2 = SXX \cdot SY - SX \cdot SXY$$

- Коэффициенты:

$$a = \frac{\Delta_1}{\Delta}$$

$$b = \frac{\Delta_2}{\Delta}$$

- Квадратичная аппроксимация:

$$P_2(x) = a_0 + a_1x + a_2x^2$$

- Система:

$$\begin{cases} n \cdot a_0 + SX \cdot a_1 + SXX \cdot a_2 = SY \\ SX \cdot a_0 + SXX \cdot a_1 + SX^3 \cdot a_2 = SXY \\ SXX \cdot a_0 + SX^3 \cdot a_1 + SX^4 \cdot a_2 = SX^2Y \end{cases}$$

- Среднеквадратичное отклонение:

$$\delta = \sqrt{\frac{\sum_{i=1}^n (\varphi(x_i) - y_i)^2}{n}}$$

Наилучшим считается уравнение, для которого значение δ минимально.

- Мера отклонения:

$$S = \sum_{i=1}^n [\varphi(x_i) - y_i]^2$$

- Коэффициент корреляции Пирсона:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 (y_i - \bar{y})^2}}, \text{ где}$$

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n}, \bar{y} = \frac{\sum_{i=1}^n y_i}{n}$$

- Интерпретация коэффициента:

При $r = \pm 1$ — строгая линейная функциональная зависимость в зависимости от знака коэффициента a в уравнении $f = ax + b$.

При $r = 0$ — связь между переменными отсутствует (в данном случае линейная).

$r < 0,3$ — связь слабая

$r = 0,3 \div 0,5$ — связь умеренная

$r = 0,5 \div 0,7$ — связь заметная

$r = 0,7 \div 0,9$ — связь высокая

$r = 0,9 \div 0,99$ — связь весьма высокая

- Коэффициент детерминации:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \varphi_i)^2}{\sum_{i=1}^n (y_i - \bar{\varphi}_i)^2}, \bar{\varphi}_i = \frac{1}{n} \sum_{i=1}^n \varphi_i$$

- Интерпретация коэффициента:

Чем ближе значение R^2 к единице ($R^2 \rightarrow 1$), тем надежнее функция φ аппроксимирует исследуемый процесс.

Если $R^2 \geq 0,95$, то говорят о высокой точности аппроксимации (модель хорошо описывает явление).

Если $0,75 \leq R^2 < 0,95$, то говорят об удовлетворительной аппроксимации (модель в целом адекватно описывает явление).

Если $0,5 \leq R^2 < 0,75$, то говорят о слабой аппроксимации (модель слабо описывает явление).

Если $R^2 < 0,5$, то говорят, что точность аппроксимации недостаточна и модель требует изменения.

Основные этапы выполнения

1. Вычислительная реализация задачи

$$y = \frac{6x}{x^4 + 5}, x \in [0; 2]; h = 0,2$$

1) Таблица табулирования функции $y = \frac{6x}{x^4 + 5}$ на интервале $x \in [0; 2]$

№	1	2	3	4	5	6	7	8	9	10	11
x_i	0	0,2	0,4	0,6	0,8	1	1,2	1,4	1,6	1,8	2
y_i	0	0,239	0,478	0,702	0,887	1	1,018	0,950	0,831	0,697	0,571

2) Линейное и квадратичное приближения по 11 точкам заданного интервала

2.1) Линейное приближение

▪ Суммы:

$$SX = 11$$

$$SXX = 15,4$$

$$SY = 7,379$$

$$SXY = 8,648$$

▪ Система:

$$\begin{cases} 15,4a + 11b = 8,648 \\ 11a + 11b = 7,374 \end{cases}$$

▪ методом Крамера:

$$\Delta = 15,4 \cdot 11 - 11 \cdot 11 = 48,4$$

$$\Delta_1 = 8,6481 \cdot 11 - 11 \cdot 7,3738 = 14,017$$

$$\Delta_2 = 15,4 \cdot 7,3738 - 11 \cdot 8,6481 = 18,427$$

▪ Коэффициенты:

$$a = \frac{14,017}{48,4} = 0,290$$

$$b = \frac{18,427}{48,4} = 0,381$$

▪ **Линейная модель:**

$$P_1(x) = 0,280x + 0,381$$

2.2) Квадратичное приближение

▪ Суммы:

$$SX^3 = 24,2$$

$$SX^4 = 40,533$$

$$SY = 7,374$$

$$SXY = 8,648$$

$$SX^2Y = 11,905$$

▪ Система:

$$\begin{cases} 11a_0 + 11a_1 + 15,4a_2 = 7,374 \\ 11a_0 + 15,4a_1 + 24,2a_2 = 8,648 \\ 15,4a_0 + 24,2a_1 + 40,533a_2 = 11,905 \end{cases}$$

▪ Коэффициенты эмпирической функции:

$$\begin{cases} a_0 = -0,042 \\ a_1 = 1,698 \\ a_2 = -0,704 \end{cases}$$

▪ Квадратичная модель:

$$P_2(x) = -0,042 + 1,698x - 0,704x^2$$

3) Среднеквадратические отклонения для каждой аппроксимирующей функции

3.1) Линейная модель $P_1(x) = 0,290x + 0,381$

№	1	2	3	4	5	6	7	8	9	10	11
x_i	0	0,2	0,4	0,6	0,8	1	1,2	1,4	1,6	1,8	2
$\varphi(x_i)$	0,381	0,439	0,497	0,555	0,612	0,670	0,728	0,786	0,844	0,902	0,960
y_i	0	0,239	0,478	0,702	0,887	1	1,018	0,950	0,831	0,697	0,571
$(\varphi(x_i) - y_i)^2$	0,145	0,040	0,0004	0,022	0,075	0,109	0,084	0,027	0,0002	0,042	0,151

$$\delta_1 = 0,251$$

3.2) Квадратичная модель $P_2(x) = -0,042 + 1,698x - 0,704x^2$

№	1	2	3	4	5	6	7	8	9	10	11
x_i	0	0,2	0,4	0,6	0,8	1	1,2	1,4	1,6	1,8	2
$\varphi(x_i)$	-0,042	0,292	0,585	0,837	1,048	1,218	1,347	1,434	1,481	1,486	1,450
y_i	0	0,239	0,478	0,702	0,887	1	1,018	0,950	0,831	0,697	0,571

$(\varphi(x_i) - y_i)^2$	0,002	0,003	0,011	0,018	0,026	0,047	0,108	0,234	0,423	0,623	0,774
--------------------------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

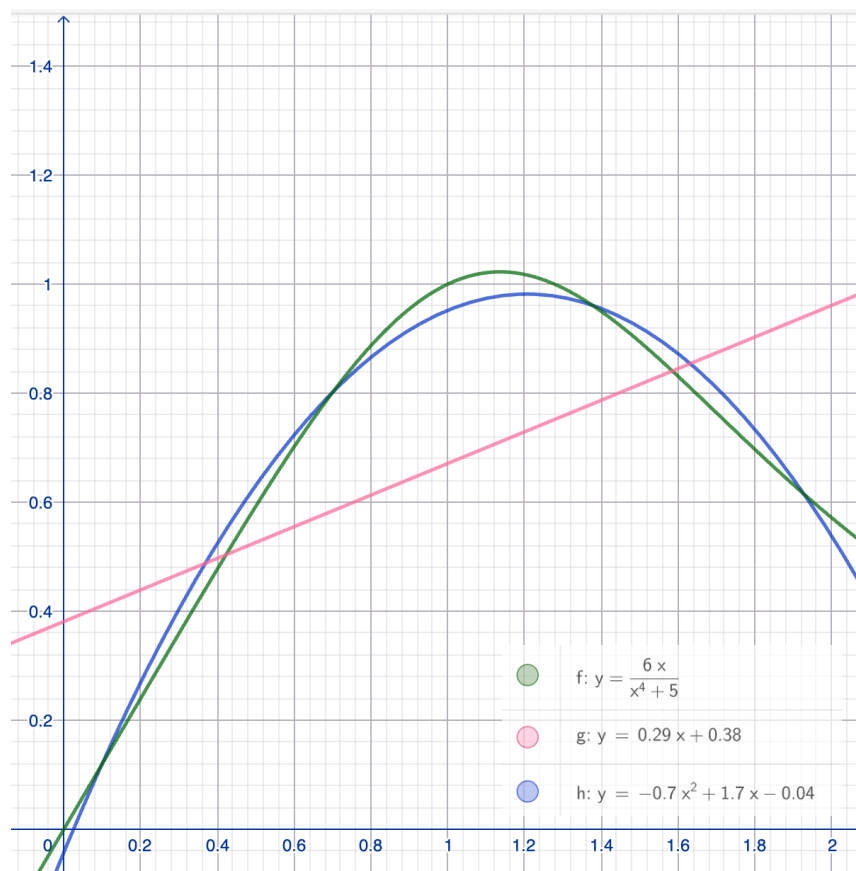
$$\delta_2 = 0,035$$

4) Наилучшее приближение

$$\delta_1 > \delta_2 \text{ (0,251 > 0,035)}$$

$\Rightarrow$ квадратичная аппроксимация наилучшее приближение,
так как у нее среднеквадратичное отклонение меньше

5) Графики функции, линейного и квадратичного приближений



2. Программная реализация задачи

```
3. import math
import sys
import matplotlib.pyplot as plt

def gauss(A, b):
    n = len(b)
    M = [A[i][:] + [b[i]] for i in range(n)]

    for col in range(n):
        max_row = max(range(col, n), key=lambda r: abs(M[r][col]))
        if abs(M[max_row][col]) < 1e-12:
            return None
        M[col], M[max_row] = M[max_row], M[col]

        pivot = M[col][col]
        M[col] = [v / pivot for v in M[col]]

        for row in range(n):
            if row == col:
                continue
            factor = M[row][col]
            M[row] = [M[row][j] - factor * M[col][j] for j in range(n
+ 1)]

    return [M[i][n] for i in range(n)]

def sums(xs, ys, max_pow=4):
    n = len(xs)
    sx = [sum(x ** k for x in xs) for k in range(max_pow + 1)]
    sxy = [sum((x ** k) * y for x, y in zip(xs, ys)) for k in
range(max_pow + 1)]
    return n, sx, sxy

def deviation_S(ys, phi):
    return sum((p - y) ** 2 for p, y in zip(phi, ys))

def std_dev(ys, phi):
    return math.sqrt(deviation_S(ys, phi) / len(ys))

def r2_score(ys, phi):
    y_mean = sum(ys) / len(ys)
    ss_res = sum((y - p) ** 2 for y, p in zip(ys, phi))
    ss_tot = sum((y - y_mean) ** 2 for y in ys)
    if abs(ss_tot) < 1e-15:
        return 1.0
    return 1.0 - ss_res / ss_tot
```

```

def pearson_r(xs, ys):
    n = len(xs)
    xm = sum(xs) / n
    ym = sum(ys) / n
    num = sum((x - xm) * (y - ym) for x, y in zip(xs, ys))
    denom = math.sqrt(sum((x - xm) ** 2 for x in xs) *
                       sum((y - ym) ** 2 for y in ys))

    if abs(denom) < 1e-15:
        return 0.0
    return num / denom


def interpret_pearson(r):
    ar = abs(r)
    if ar < 0.3: return "слабая"
    if ar < 0.5: return "умеренная"
    if ar < 0.7: return "заметная"
    if ar < 0.9: return "высокая"
    if ar < 1.0: return "весьма высокая"
    return "строгая линейная функциональная"


def interpret_r2(r2):
    if r2 >= 0.95: return "высокая точность аппроксимации (модель хорошо описывает явление)"
    if r2 >= 0.75: return "удовлетворительная аппроксимация (модель в целом адекватно описывает явление)"
    if r2 >= 0.5: return "слабая аппроксимация (модель слабо описывает явление)"
    return "точность аппроксимации недостаточна, модель требует изменения"


class ApproxResult:
    def __init__(self, name, formula, coeffs, phi, xs, ys):
        self.name = name
        self.formula = formula
        self.coeffs = coeffs
        self.phi = phi
        self.eps = [p - y for p, y in zip(phi, ys)]
        self.S = deviation_S(ys, phi)
        self.delta = std_dev(ys, phi)
        self.R2 = r2_score(ys, phi)
        self.xs = xs
        self.ys = ys


def linear(xs, ys):
    n, sx, sxy = sums(xs, ys, 2)
    A = [[sx[2], sx[1]], [sx[1], n]]
    b = [sxy[1], sxy[0]]
    sol = gauss(A, b)
    if sol is None:
        return None
    a, b_coef = sol
    phi = [a * x + b_coef for x in xs]
    return ApproxResult("Линейная", f" $\phi(x) = \{a:.4f\} \cdot x + \{b\_coef:.4f\}$ ",
                        {"a": a, "b": b_coef}, phi, xs, ys)


def quadratic(xs, ys):
    n, sx, sxy = sums(xs, ys, 4)

```

```

A = [
    [n, sx[1], sx[2]],
    [sx[1], sx[2], sx[3]],
    [sx[2], sx[3], sx[4]],
]
b = [sxy[0], sxy[1], sxy[2]]
sol = gauss(A, b)
if sol is None:
    return None
a0, a1, a2 = sol
phi = [a0 + a1 * x + a2 * x ** 2 for x in xs]
return ApproxResult("Квадратичная",
                    f"φ(x) = {a0:.4f} + {a1:.4f} · x +",
                    ({a2:.4f}) · x²",
                    {"a0": a0, "a1": a1, "a2": a2}, phi, xs, ys)

def cubic(xs, ys):
    n, sx, sxy = sums(xs, ys, 6)
    A = [
        [n, sx[1], sx[2], sx[3]],
        [sx[1], sx[2], sx[3], sx[4]],
        [sx[2], sx[3], sx[4], sx[5]],
        [sx[3], sx[4], sx[5], sx[6]],
    ]
    b = [sxy[0], sxy[1], sxy[2], sxy[3]]
    sol = gauss(A, b)
    if sol is None:
        return None
    a0, a1, a2, a3 = sol
    phi = [a0 + a1 * x + a2 * x ** 2 + a3 * x ** 3 for x in xs]
    return ApproxResult("Кубическая",
                        f"φ(x) = {a0:.4f} + {a1:.4f} · x +",
                        ({a2:.4f}) · x² + ({a3:.4f}) · x³",
                        {"a0": a0, "a1": a1, "a2": a2, "a3": a3},
                        phi, xs, ys)

def exponential(xs, ys):
    pairs = [(x, y) for x, y in zip(xs, ys) if y > 0]
    if len(pairs) < 2:
        return None
    xs2 = [p[0] for p in pairs]
    lny = [math.log(p[1]) for p in pairs]
    res = linear(xs2, lny)
    if res is None:
        return None
    b_coef = res.coefs["a"]
    a = math.exp(res.coefs["b"])
    phi = [a * math.exp(b_coef * x) for x in xs]
    return ApproxResult("Экспоненциальная",
                        f"φ(x) = {a:.4f} · e^{({b_coef:.4f}) · x}",
                        {"a": a, "b": b_coef}, phi, xs, ys)

def logarithmic(xs, ys):
    pairs = [(x, y) for x, y in zip(xs, ys) if x > 0]
    if len(pairs) < 2:
        return None
    lnx = [math.log(p[0]) for p in pairs]
    ys2 = [p[1] for p in pairs]
    res = linear(lnx, ys2)
    if res is None:
        return None

```

```

a = res.coef["a"]
b_coef = res.coef["b"]
phi = []
for x in xs:
    if x > 0:
        phi.append(a * math.log(x) + b_coef)
    else:
        phi.append(float("nan"))
valid = [(p, y) for p, y in zip(phi, ys) if not math.isnan(p)]
if not valid:
    return None
phi_v = [v[0] for v in valid]
ys_v = [v[1] for v in valid]
eps = [p - y for p, y in zip(phi_v, ys_v)]
S = sum(e ** 2 for e in eps)
delta = math.sqrt(S / len(ys_v))
R2 = r2_score(ys_v, phi_v)

r = ApproxResult("Логарифмическая",
                  f"φ(x) = {a:.4f} · ln(x) + ({b_coef:.4f})",
                  {"a": a, "b": b_coef}, phi, xs, ys)

r.eps = [p - y for p, y in zip(phi, ys)]
r.S = S
r.delta = delta
r.R2 = R2
return r

def power(xs, ys):
    pairs = [(x, y) for x, y in zip(xs, ys) if x > 0 and y > 0]
    if len(pairs) < 2:
        return None
    lnx = [math.log(p[0]) for p in pairs]
    lny = [math.log(p[1]) for p in pairs]
    res = linear(lnx, lny)
    if res is None:
        return None
    b_coef = res.coef["a"]
    a = math.exp(res.coef["b"])
    phi = []
    for x in xs:
        if x > 0:
            phi.append(a * (x ** b_coef))
        else:
            phi.append(float("nan"))
    valid = [(p, y) for p, y in zip(phi, ys) if not math.isnan(p)]
    if not valid:
        return None
    phi_v = [v[0] for v in valid]
    ys_v = [v[1] for v in valid]
    S = sum((p - y) ** 2 for p, y in zip(phi_v, ys_v))
    delta = math.sqrt(S / len(ys_v))
    R2 = r2_score(ys_v, phi_v)

    r = ApproxResult("Степенная",
                      f"φ(x) = {a:.4f} · x^({b_coef:.4f})",
                      {"a": a, "b": b_coef}, phi, xs, ys)

    r.S = S
    r.delta = delta
    r.R2 = R2
    return r

```

```

def input_from_console():
    while True:
        try:
            n = int(input("Введите количество точек (8-12): "))
            if 8 <= n <= 12:
                break
            print("Ошибка: допустимо от 8 до 12 точек")
        except ValueError:
            print("Ошибка: введите целое число")

    xs, ys = [], []
    print("Введите значения x и y через пробел (по одной паре в строке):")
    for i in range(n):
        while True:
            try:
                line = input(f"    Точка {i + 1}: ").split()
                xs.append(float(line[0].replace(",", ".")))
                ys.append(float(line[1].replace(",", ".")))
                break
            except (ValueError, IndexError):
                print("    Ошибка. Введите два числа через пробел")
    return xs, ys

def input_from_file(path):
    xs, ys = [], []
    with open(path, encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            parts = line.replace(",", ".").split()
            if len(parts) >= 2:
                xs.append(float(parts[0]))
                ys.append(float(parts[1]))
    n = len(xs)
    if not (8 <= n <= 12):
        raise ValueError(f"Файл содержит {n} точек — допустимо от 8 до 12")
    return xs, ys

def validate_data(xs, ys):
    n = len(xs)
    if n != len(ys):
        raise ValueError("Количество x и y не совпадает")
    if not (8 <= n <= 12):
        raise ValueError(f"Допустимо от 8 до 12 точек, получено {n}")
    if len(set(xs)) != n:
        raise ValueError("Значения x должны быть уникальными")

SEPARATOR = "-" * 80

def print_table(res: ApproxResult):
    header = f"{'i':>3} | {'xi':>8} | {'yi':>10} | {'φ(xi)':>10} | {'εi':>10}"
    print(header)
    print("-" * len(header))
    for i, (x, y, p, e) in enumerate(zip(res.xs, res.ys, res.phi, res.eps), 1):
        p_str = f"{p:10.6f}" if not math.isnan(p) else f"{'н/д':>10}"

```

```

        e_str = f"{e:10.6f}" if not math.isnan(e) else f"{'н/д':>10}"
        print(f"{i:>3} | {x:8.4f} | {y:10.6f} | {p_str} | {e_str}")

def print_result(res: ApproxResult, xs, ys, print_pearson=False):
    print(f"\n{'=' * 80}")
    print(f"    {res.name.upper()} АППРОКСИМАЦИЯ")
    print(f"    Формула: {res.formula}")
    print()
    print("    Коэффициенты:")
    for k, v in res.coeffs.items():
        print(f"        {k} = {v:.6f}")
    print()
    print_table(res)
    print()
    print(f"    S    (мера отклонения)                = {res.S:.6f}")
    print(f"     $\delta$     (среднеквадратичное откл.) = {res.delta:.6f}")
    print(f"    R2    (коэффициент детерминации) = {res.R2:.6f}")
    print(f"    Интерпретация R2: {interpret_r2(res.R2)}")

    if print_pearson:
        r = pearson_r(xs, ys)
        print()
        print(f"    Коэффициент корреляции Пирсона r = {r:.6f}")
        print(f"    Интерпретация: связь {interpret_pearson(r)}")

def plot_results(results, xs, ys, best):
    fig, ax = plt.subplots(1, 1, figsize=(10, 6))

    x_min = min(xs)
    x_max = max(xs)
    x_range = x_max - x_min
    x_min_plot = x_min - 0.05 * x_range
    x_max_plot = x_max + 0.05 * x_range

    ax.scatter(xs, ys, color='black', s=60, zorder=5, label='Исходные  
данные',
               edgecolors='white', linewidth=1.5)

    colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
              '#8c564b']

    for idx, res in enumerate(results):
        x_plot = [x_min_plot + i * (x_max_plot - x_min_plot) / 199
                  for i in range(200)]
        y_plot = []
        for x in x_plot:
            try:
                y_val = _eval_approx(res, x)
                y_plot.append(y_val if y_val is not None and not
                              math.isnan(y_val) else None)
            except:
                y_plot.append(None)

        ax.plot(x_plot, y_plot, color=colors[idx % len(colors)],
                linewidth=2, label=res.name, zorder=3)

    ax.set_xlim(x_min_plot, x_max_plot)

    ax.margins(y=0.1)

    ax.set_xlabel('x', fontsize=11)
    ax.set_ylabel('y', fontsize=11)

```



```

ax.set_title('Аппроксимация функции методом наименьших
квадратов',
             fontsize=13, fontweight='bold', pad=15)

ax.legend(loc='best', fontsize=9)
ax.grid(True, alpha=0.3, linestyle='--')

plt.tight_layout()
plt.show()

def _eval_approx(res: ApproxResult, x):
    c = res.coeffs
    name = res.name
    if name == "Линейная":
        return c["a"] * x + c["b"]
    if name == "Квадратичная":
        return c["a0"] + c["a1"] * x + c["a2"] * x ** 2
    if name == "Кубическая":
        return c["a0"] + c["a1"] * x + c["a2"] * x ** 2 + c["a3"] * x
    if name == "Экспоненциальная":
        return c["a"] * math.exp(c["b"] * x)
    if name == "Логарифмическая":
        if x <= 0:
            return None
        return c["a"] * math.log(x) + c["b"]
    if name == "Степенная":
        if x <= 0:
            return None
        return c["a"] * (x ** c["b"])
    return None

def save_to_file(path, results, xs, ys, best):
    import io, sys
    buf = io.StringIO()
    old_stdout = sys.stdout
    sys.stdout = buf

    print("Аппроксимация функции методом наименьших квадратов")
    print("=" * 80)
    for res in results:
        print_result(res, xs, ys, print_pearson=(res.name ==
"Линейная"))

    print(f"\n{'=' * 80}")
    print(f"    НАИЛУЧШАЯ АППРОКСИМАЦИЯ: {best.name.upper()}")
    print(f"    Формула : {best.formula}")
    print(f"     $\delta$  (min) = {best.delta:.6f}")
    print(f"    R2      = {best.R2:.6f} →
{interpret_r2(best.R2)}")

    sys.stdout = old_stdout
    with open(path, "w", encoding="utf-8") as f:
        f.write(buf.getvalue())
    print(f"\n    Результаты сохранены в файл: {path}")

def main():
    while True:
        choice = input("Ваш выбор [1/2]: ").strip()
        if choice == "1":
            xs, ys = input_from_console()

```

```

        break
    elif choice == "2":
        path = input("Путь к файлу: ").strip()
        try:
            xs, ys = input_from_file(path)
            break
        except (FileNotFoundError, ValueError) as e:
            print(f"Ошибка: {e}")
            sys.exit(1)
    else:
        print("Неверный выбор. Введите 1/2")

    try:
        validate_data(xs, ys)
    except ValueError as e:
        print(f"\nОшибка в данных: {e}")
        sys.exit(1)

    print(f"\n Загружено {len(xs)} точек.")
    print(f"  x: {[round(v, 4) for v in xs]}")
    print(f"  y: {[round(v, 6) for v in ys]}")

    funcs = [
        ("Линейная", linear),
        ("Квадратичная", quadratic),
        ("Кубическая", cubic),
        ("Экспоненциальная", exponential),
        ("Логарифмическая", logarithmic),
        ("Степенная", power),
    ]

    results = []
    for name, func in funcs:
        try:
            res = func(xs, ys)
            if res is None:
                print(f"\n [{name}] - не удалось построить (данные не подходят для этого типа)")
            else:
                results.append(res)
        except Exception as e:
            print(f"\n [{name}] - ошибка: {e}")

    if not results:
        print("Ни одна аппроксимация не была успешно построена")
        sys.exit(1)

    for res in results:
        print_result(res, xs, ys, print_pearson=(res.name == "Линейная"))

    best = min(results, key=lambda r: r.delta)
    print(f"\n{'=' * 80}")
    print(f"  НАИЛУЧШАЯ АППРОКСИМАЦИЯ: {best.name.upper()}")
    print(f"  Формула : {best.formula}")
    print(f"   $\delta$  (min) = {best.delta:.6f}")
    print(f"   $R^2$  = {best.R2:.6f} → {interpret_r2(best.R2)}")

    plot_results(results, xs, ys, best)

    try:
        save_ans = input("\nСохранить результаты в файл? [y/n]: ")
        save_ans = save_ans.strip().lower()

```

```

        if save_ans == "y":
            out_path = input("Путь к файлу (Enter = results.txt):
            ").strip() or "results.txt"
            save_to_file(out_path, results, xs, ys, best)
        except EOFError:
            pass

if __name__ == "__main__":
    main()

```

Вывод программы

Ваш выбор [1/2]: 2

Путь к файлу: data.txt

Загружено 11 точек

x: [0.0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.4, 1.6, 1.8, 2.0]

y: [0.0, 0.239, 0.478, 0.702, 0.887, 1.0, 1.018, 0.95, 0.831, 0.697, 0.571]

ЛИНЕЙНАЯ АППРОКСИМАЦИЯ

Формула: $\varphi(x) = 0.2897 \cdot x + (0.3806)$

Коэффициенты:

a = 0.289682

b = 0.380591

i	xi	yi	$\varphi(x_i)$	ε_i
1	0.0000	0.000000	0.380591	0.380591
2	0.2000	0.239000	0.438527	0.199527
3	0.4000	0.478000	0.496464	0.018464
4	0.6000	0.702000	0.554400	-0.147600
5	0.8000	0.887000	0.612336	-0.274664
6	1.0000	1.000000	0.670273	-0.329727
7	1.2000	1.018000	0.728209	-0.289791
8	1.4000	0.950000	0.786145	-0.163855
9	1.6000	0.831000	0.844082	0.013082
10	1.8000	0.697000	0.902018	0.205018
11	2.0000	0.571000	0.959955	0.388955

S (мера отклонения) = 0.695264

δ (среднеквадратичное откл.) = 0.251408

R^2 (коэффициент детерминации) = 0.346859

Интерпретация R^2 : точность аппроксимации недостаточна, модель требует изменения

Коэффициент корреляции Пирсона $r = 0.588947$

Интерпретация: связь заметная

КВАДРАТИЧНАЯ АППРОКСИМАЦИЯ

Формула: $\varphi(x) = -0.0422 + 1.6991 \cdot x + (-0.7047) \cdot x^2$

Коэффициенты:

$$a_0 = -0.042224$$

$$a_1 = 1.699064$$

$$a_2 = -0.704691$$

i	x _i	y _i	φ(x _i)	ε _i
1	0.0000	0.000000	-0.042224	-0.042224
2	0.2000	0.239000	0.269401	0.030401
3	0.4000	0.478000	0.524651	0.046651
4	0.6000	0.702000	0.723526	0.021526
5	0.8000	0.887000	0.866025	-0.020975
6	1.0000	1.000000	0.952149	-0.047851
7	1.2000	1.018000	0.981898	-0.036102
8	1.4000	0.950000	0.955271	0.005271
9	1.6000	0.831000	0.872269	0.041269
10	1.8000	0.697000	0.732892	0.035892
11	2.0000	0.571000	0.537140	-0.033860

$$S \text{ (мера отклонения)} = 0.013546$$

$$\delta \text{ (среднеквадратичное откл.)} = 0.035091$$

$$R^2 \text{ (коэффициент детерминации)} = 0.987275$$

Интерпретация R^2 : высокая точность аппроксимации (модель хорошо описывает явление)

КУБИЧЕСКАЯ АППРОКСИМАЦИЯ

$$\text{Формула: } \varphi(x) = -0.0358 + 1.6478 \cdot x + (-0.6375) \cdot x^2 + (-0.0224) \cdot x^3$$

Коэффициенты:

$$a_0 = -0.035776$$

$$a_1 = 1.647842$$

$$a_2 = -0.637529$$

$$a_3 = -0.022387$$

i	x _i	y _i	φ(x _i)	ε _i
1	0.0000	0.000000	-0.035776	-0.035776
2	0.2000	0.239000	0.268112	0.029112
3	0.4000	0.478000	0.519923	0.041923
4	0.6000	0.702000	0.718583	0.016583
5	0.8000	0.887000	0.863016	-0.023984
6	1.0000	1.000000	0.952149	-0.047851
7	1.2000	1.018000	0.984907	-0.033093
8	1.4000	0.950000	0.960214	0.010214
9	1.6000	0.831000	0.876998	0.045998
10	1.8000	0.697000	0.734182	0.037182
11	2.0000	0.571000	0.530692	-0.040308

$$S \text{ (мера отклонения)} = 0.013347$$

$$\delta \text{ (среднеквадратичное откл.)} = 0.034834$$

$$R^2 \text{ (коэффициент детерминации)} = 0.987461$$

Интерпретация R^2 : высокая точность аппроксимации (модель хорошо описывает явление)

ЭКСПОНЕНЦИАЛЬНАЯ АППРОКСИМАЦИЯ

Формула: $\varphi(x) = 0.4662 \cdot e^{(0.3499 \cdot x)}$

Коэффициенты:

$a = 0.466188$

$b = 0.349871$

i	x_i	y_i	$\varphi(x_i)$	ε_i
1	0.0000	0.000000	0.466188	0.466188
2	0.2000	0.239000	0.499978	0.260978
3	0.4000	0.478000	0.536216	0.058216
4	0.6000	0.702000	0.575081	-0.126919
5	0.8000	0.887000	0.616764	-0.270236
6	1.0000	1.000000	0.661467	-0.338533
7	1.2000	1.018000	0.709410	-0.308590
8	1.4000	0.950000	0.760829	-0.189171
9	1.6000	0.831000	0.815974	-0.015026
10	1.8000	0.697000	0.875116	0.178116
11	2.0000	0.571000	0.938545	0.367545

S (мера отклонения) = 0.790624

δ (среднеквадратичное откл.) = 0.268095

R^2 (коэффициент детерминации) = 0.257276

Интерпретация R^2 : точность аппроксимации недостаточна, модель требует изменения

ЛОГАРИФМИЧЕСКАЯ АППРОКСИМАЦИЯ

Формула: $\varphi(x) = 0.2175 \cdot \ln(x) + (0.7588)$

Коэффициенты:

$a = 0.217545$

$b = 0.758836$

i	x_i	y_i	$\varphi(x_i)$	ε_i
1	0.0000	0.000000	н/д	н/д
2	0.2000	0.239000	0.408710	0.169710
3	0.4000	0.478000	0.559501	0.081501
4	0.6000	0.702000	0.647709	-0.054291
5	0.8000	0.887000	0.710292	-0.176708
6	1.0000	1.000000	0.758836	-0.241164
7	1.2000	1.018000	0.798499	-0.219501
8	1.4000	0.950000	0.832034	-0.117966
9	1.6000	0.831000	0.861083	0.030083
10	1.8000	0.697000	0.886707	0.189707
11	2.0000	0.571000	0.909627	0.338627

S (мера отклонения) = 0.341436

δ (среднеквадратичное откл.) = 0.184780

R^2 (коэффициент детерминации) = 0.401305

Интерпретация R^2 : точность аппроксимации недостаточна, модель требует изменения

СТЕПЕННАЯ АППРОКСИМАЦИЯ

Формула: $\varphi(x) = 0.7153 \cdot x^{(0.4366)}$

Коэффициенты:

$a = 0.715276$

$b = 0.436590$

i	x_i	y_i	$\varphi(x_i)$	ε_i
1	0.0000	0.000000	н/д	н/д
2	0.2000	0.239000	0.354250	0.115250
3	0.4000	0.478000	0.479443	0.001443
4	0.6000	0.702000	0.572290	-0.129710
5	0.8000	0.887000	0.648879	-0.238121
6	1.0000	1.000000	0.715276	-0.284724
7	1.2000	1.018000	0.774539	-0.243461
8	1.4000	0.950000	0.828460	-0.121540
9	1.6000	0.831000	0.878193	0.047193
10	1.8000	0.697000	0.924534	0.227534
11	2.0000	0.571000	0.968055	0.397055

S (мера отклонения) = 0.453576

δ (среднеквадратичное откл.) = 0.212973

R^2 (коэффициент детерминации) = 0.204672

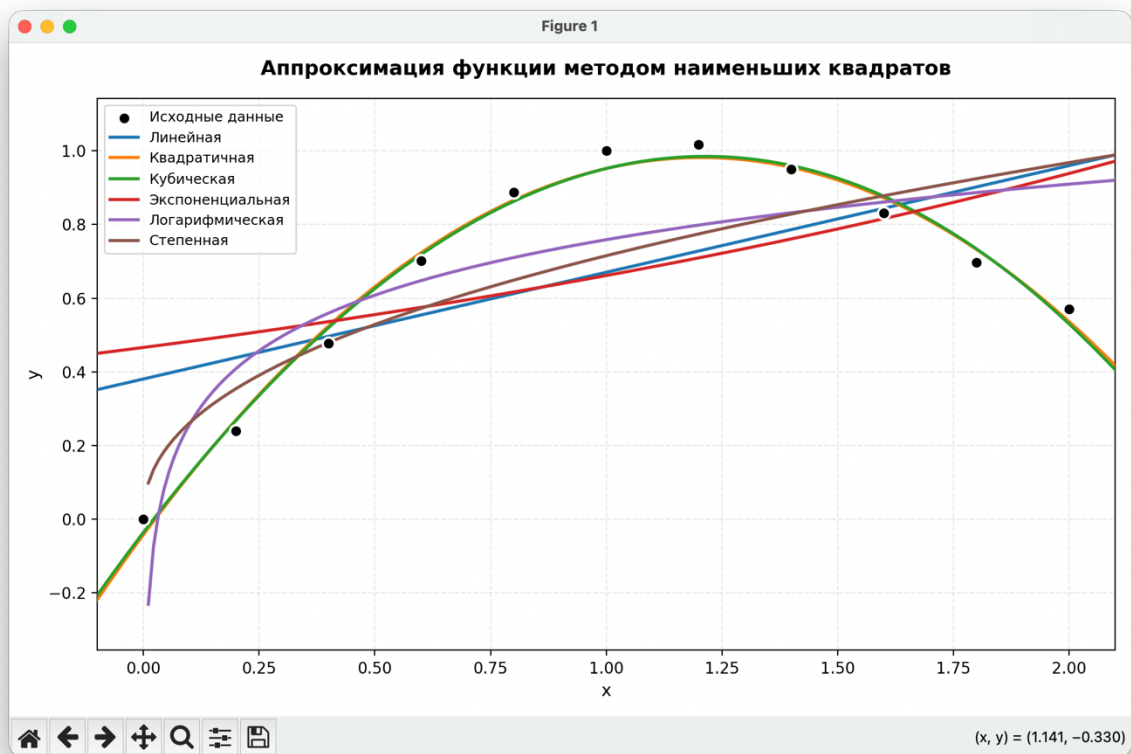
Интерпретация R^2 : точность аппроксимации недостаточна, модель требует изменения

НАИЛУЧШАЯ АППРОКСИМАЦИЯ: КУБИЧЕСКАЯ

Формула : $\varphi(x) = -0.0358 + 1.6478 \cdot x + (-0.6375) \cdot x^2 + (-0.0224) \cdot x^3$

$\delta(\min) = 0.034834$

$R^2 = 0.987461 \rightarrow$ высокая точность аппроксимации (модель хорошо описывает явление)



Сохранить результаты в файл? [y/n]: y

Путь к файлу (Enter = results.txt):

Результаты сохранены в файл: results.txt

Вывод

Выполнение лабораторной работы по вычислительной математике позволило мне познакомиться с методом наименьших квадратов и его применением для аппроксимации табличных значений. Данная работа дала мне возможность применить полученные теоретические знания на практике, развить навыки работы с информацией. В процессе выполнения я вручную воспользовалась линейным и квадратическим приближением для МНК и написала программу, реализующую МНК для различных типов функции (линейной, полиномиальной 2-й и 3-й степени, экспоненциальной, логарифмической и степенной), на ЯП Python. Приобретённый опыт поможет мне при дальнейшем изучении предмета вычислительная математика.